CSCI 6600 — FALL 2026

# Database Management System Types

A comprehensive guide — 17 DBMS types explored

| 01 | 18 | 120 min |
| --- | --- | --- |
| Lecture | Types Covered | Duration |

Instructor
**Brian Dietrick**

Department of Computer Science · East Carolina University

# Relational Databases



## Typical Uses

- Transactional systems (banking, e-commerce)
- ERP and CRM platforms
- Financial records
- HR management

- …pretty much everything has been placed in a relational database at one time or another

- Also known as tabular or rectangular data

## Pros

- ACID compliance and strong data integrity
- Mature ecosystem with excellent tooling
- Powerful SQL query language
- Well-understood normalization

## Cons

- Rigid schema makes changes costly
- Poor horizontal scalability
- Object-relational impedance mismatch
- Complex JOINs at scale

- Relational databases were designed for **shared state**.
- ACID forces shared state.
- Shared state forces coordination.
- Coordination kills horizontal scalability.

## Examples

**PostgreSQL** **MySQL** **Oracle Database** **Microsoft SQL Server** **SQLite**

> *Relational databases organize data into structured tables - the foundational model for most enterprise applications, built on decades of maturity.*

# Key-Value Databases



### Typical Uses

- Session management
- Caching layers
- Shopping cart storage
- Leaderboards
- Real-time configuration

### Pros

- Extremely fast O(1) lookups
- Horizontally scalable by design
- Simple data model
- Ideal for caching layers

### Cons

- No query language beyond key lookup
- Limited data modeling
- Difficult to query across values
- No native relationship support

### Examples

**Redis** **Amazon DynamoDB** **Memcached** **Riak** **Aerospike**

*Key-value stores are the simplest and fastest database model - a hash map at scale, optimized for high-throughput lookups.*

# Document Databases



**TYPICAL USES**

- Content management systems
- User profiles
- Product catalogs
- Mobile app backends
- IoT event storage

**PROS**

- Flexible schema per document
- JSON/BSON maps naturally to application objects
- Rich querying within documents
- Horizontal scaling with sharding

**CONS**

- No cross-document transactions in all implementations
- Data duplication via denormalization
- Harder to enforce consistency

**EXAMPLES**

- **MongoDB**
- **CouchDB**
- **Amazon DocumentDB**
- **Firestore**
- **RavenDB**
- **LiteDB**

Document databases store self-contained JSON-like records - ideal for applications with evolving, hierarchical data structures.

# Column-Family Databases



## TYPICAL USES

- IoT sensor streams
- Audit logs
- Recommendation engines
- Large-scale analytics
- Write-heavy workloads

## PROS

- Optimized for write-heavy workloads
- Efficient column-level compression
- Massive horizontal scalability
- Tunable consistency

## CONS

- Complex data modeling (query-first design)
- Limited ad-hoc query support
- Steep learning curve
- No JOINs

## EXAMPLES

- Apache Cassandra
- Apache HBase
- Google Bigtable
- ScyllaDB

*Column-family databases group data by columns - purpose-built for write-intensive, high-volume workloads at global scale.*

# Graph Databases



### Typical Uses

- Social networks
- Fraud detection
- Recommendation engines
- Knowledge graphs
- IT topology mapping

### Pros

- Native relationship traversal — no costly JOINs
- Intuitive modeling of connected data
- Excellent multi-hop queries
- Flexible property graph model

### Cons

- Poor performance for non-graph workloads
- Limited ecosystem
- Scaling challenges for very large graphs
- Steeper learning curve

### Examples

**Neo4j**

**Amazon Neptune**

**ArangoDB**

**JanusGraph**

**TigerGraph**

**SQL Server**

*Graph databases treat relationships as first-class citizens - making complex network traversals fast and intuitive to model.*

# NewSQL Databases



**Typical Uses**
- Financial transactions at cloud scale
- Global e-commerce
- Distributed SaaS applications
- Multi-region ACID workloads

**Pros**
- Full ACID compliance at distributed scale
- Familiar SQL interface
- Horizontal scalability of NoSQL
- Strong consistency

**Cons**
- Higher operational complexity
- Newer and smaller ecosystem
- Higher cost at small scale
- Vendor lock-in risk

**Examples**
- Google Spanner
- CockroachDB
- YugabyteDB
- TiDB
- VoltDB

*NewSQL combines relational ACID guarantees with NoSQL horizontal scalability - the best of both worlds for modern cloud applications.*

# Time-Series Databases



## Typical Uses

- Application performance monitoring
- IoT sensor data
- Financial market data
- Infrastructure metrics
- Industrial telemetry

## Pros

- Optimized storage and compression for time-ordered data
- Fast range queries over time windows
- Built-in downsampling and retention
- High ingest throughput

## Cons

- Limited use outside time-series workloads
- Weaker general-purpose queries
- Data modeling tied to time as primary key
- Retention complexity

## Examples

**InfluxDB**

**TimescaleDB**

**Prometheus**

**OpenTSDB**

**QuestDB**

*Time-series databases are engineered for the relentless stream of timestamped data - with unmatched compression and query speed.*

# Object-Oriented Databases



**Typical Uses**
- CAD/CAM systems
- Scientific simulations
- Telecom network management
- Multimedia storage
- Complex OOP object hierarchies

**Pros**
- Eliminates object-relational impedance mismatch
- Stores complex objects natively
- Inheritance and polymorphism supported
- Natural fit for OOP languages

**Cons**
- Limited adoption and small ecosystem
- Poor SQL interoperability
- Weak query language standardization
- Not suited for analytics

**Examples**
- db4o
- ObjectDB
- Versant Object Database
- GemStone/S

*Object-oriented databases persist objects directly as they exist in code - eliminating the translation layer between application and storage.*

# Hierarchical Databases



### Typical Uses

- Legacy mainframe systems
- File system organization
- XML document storage
- Org chart data
- Bill of materials in manufacturing
- Hospital Information Systems (HIS)

### Pros

- Very fast for hierarchical navigational queries
- Simple parent-child model
- Efficient for predefined access patterns
- Proven mainframe reliability

### Cons

- Rigid tree — poor fit for many-to-many relationships
- Expensive schema changes
- Limited query flexibility
- Largely outdated for new development

### Examples

- IBM IMS
- Windows Registry
- XML databases
- LDAP directories
- Epic Chronicles

*Hierarchical databases organize data as a tree - highly efficient for known traversal paths but inflexible for varied modern queries.*

# Network Databases



### Typical Uses

- Legacy telecom systems
- Airline reservation systems
- Manufacturing resource planning
- Early ERP predecessors

### Pros

- Supports many-to-many relationships natively
- Faster navigation than relational for known paths
- Flexible record linking via pointers

### Cons

- Highly complex schema design
- Navigation requires knowing data structure upfront
- Nearly obsolete for new systems
- Minimal tooling and community

### Examples

- IDMS
- IDS (Integrated Data Store)
- TurboIMAGE
- Raima Database Manager

*Network databases extend the hierarchical model with many-to-many links - a powerful but complex precursor to the relational model.*

# Multimodal Databases



### Typical Uses

- Applications requiring multiple data models
- Microservices architectures
- Unified data platforms
- Hybrid workloads

### Pros

- Reduces infrastructure complexity
- Supports multiple data models in one system
- Lowers operational overhead
- Flexible data modeling

### Cons

- May not excel at any single model
- Complex query optimization across models
- Vendor lock-in to proprietary query languages

### Examples

**ArangoDB**

**Azure Cosmos DB**

**OrientDB**

**FaunaDB**

**Couchbase**

**SQL Server**

*Multimodel databases unify multiple paradigms under one platform - reducing operational complexity for diverse data needs.*

# In-Memory Databases



## TYPICAL USES

- Real-time bidding
- Session caching
- Leaderboards
- Gaming state
- Financial trading
- Sub-millisecond analytics

## PROS

- Microsecond read/write latency
- Eliminates disk I/O
- High throughput for read-intensive workloads
- Predictable performance

## CONS

- Data loss risk on crash without persistence
- High cost — RAM is expensive at scale
- Dataset size limited by available memory

## EXAMPLES

- Redis
- Memcached
- VoltDB
- SAP HANA
- Aerospike
- Apache Ignite

*In-memory databases keep all data in RAM - trading storage cost for the ultimate in speed, enabling microsecond-latency applications.*

# Embedded Databases



### Typical Uses

- Mobile apps
- Desktop software
- IoT devices
- Browser-based local storage
- Edge computing
- Automotive systems
- Game save files

### Pros

- No separate server process
- Zero network latency (in-process)
- Simple deployment — single file or library
- Excellent for offline-first apps

### Cons

- Not designed for concurrent multi-user access
- Limited scalability
- Smaller feature set than server databases
- Sync complexity to server

### Examples

**SQLite**

**LevelDB**

**RocksDB**

**Berkeley DB**

**Realm**

**DuckDB**

**LiteDB**

**SQL Cipher**

*Embedded databases run within the application process itself - no server, no network, just fast local data access for devices and apps.*

# Search Engine Databases



### Typical Uses

- Full-text search
- E-commerce product search
- Log aggregation
- Autocomplete
- Faceted navigation
- Enterprise search

### Pros

- Optimized for full-text and fuzzy search
- Powerful relevance scoring
- Handles unstructured text natively
- Rich aggregation capabilities

### Cons

- Not a primary data store — requires sync from source of truth
- Complex relevance tuning
- Significant index storage overhead

### Examples

Elasticsearch

Apache Solr

OpenSearch

Meilisearch

Typesense

*Search engine databases transform unstructured text into instantly searchable, relevance-ranked results at any scale.*

# Spatial Databases



### Typical Uses

- Geographic information systems (GIS)
- Mapping applications
- Location-based services
- Logistics optimization
- Real estate platforms

### Pros

- Native geometric and geographic data types
- Efficient spatial indexing (R-tree, QuadTree)
- Standard SQL/MM spatial queries
- GIS tool integration

### Cons

- Specialized knowledge required
- Performance degrades on complex operations at scale
- Enterprise GIS licensing costs

### Examples

- PostGIS
- Oracle Spatial
- SQL Server
- SpatiaLite
- MongoDB Geospatial

*Spatial databases extend storage with geometry and geography - enabling location-aware queries for maps, navigation, and geospatial analytics.*

# OLAP — Online Analytical Processing



### Typical Uses

- Business intelligence dashboards
- Data warehousing
- Financial reporting
- Sales analytics
- Trend analysis
- Executive decision support

### Pros

- Optimized for complex analytical queries
- Columnar storage for fast aggregations
- Multidimensional analysis (cubes)
- Excellent compression on analytical data

### Cons

- Not designed for transactional (OLTP) workloads
- ETL pipeline required
- High historical data storage costs
- Performance depends on schema design

### Examples

- Snowflake
- Google BigQuery
- Amazon Redshift
- Apache Druid
- ClickHouse
- DuckDB
- SQL Server – Analysis Services

*OLAP databases are the analytical engines behind business intelligence - turning terabytes of historical data into fast, multidimensional insights.*

# DFS-Backed Databases



### Typical Uses

- Big data batch processing
- Data lake storage
- Large-scale ETL pipelines
- Hadoop ecosystem workflows
- ML training data storage

### Pros

- Massive scalability on commodity hardware
- Fault-tolerant through replication
- Cost-effective for petabyte-scale
- Integrates with Spark, Hive, and big data frameworks

### Cons

- High query latency — not suited for real-time
- Complex cluster management
- Designed for batch, not interactive queries
- Overkill for small datasets

### Examples

- Apache HBase on HDFS
- Apache Hive
- Delta Lake
- Apache Iceberg
- Apache Hudi

*DFS-backed databases harness distributed file systems for petabyte-scale storage - the backbone of big data architectures and data lakes.*

# Vector Databases



### Typical Uses

- Semantic Search
- Retrieval Augmented Generation (RAG)
- Recommendation Systems
- Multimodal Search (e.g. text + audio + image + video)
- Fraud Detection and Anomaly Detection
- Long-Term Agent Memory
- Enterprise Search and Knowledge Management
- Real-Time Personalization

### Pros

- Semantic Retrieval
- High Performance at Scale (up to 1000X faster)
- Multimodal Support
- Hybrid Search
- Real-Time Recommendations
- Flexible Schema

### Cons

- Operational Cost at Scale (significant memory and CPU)
- Latency under high load (recall vs latency)
- Data Freshness Issues (update requires re-indexing)
- Accuracy Trade-offs
- Infrastructure Complexity
- Not Always Necessary

### Examples

**Dedicated Vector DBs**

- Pinecone
- Weaviate
- Qdrant
- Milvus
- Chroma

**Multimodal Vector DBs**

- ArangoDB
- Teradata Enterprise Vector Store

**Conventional DB with Vector data feature**

- PostgresSQL + pgvector
- Elasticsearch / OpenSearch
- SQL Server

*Vector DBs are fast, scalable similarity search over high-dimensional embeddings – they shine anywhere you need semantic search rather than keyword retrieval.*